[DETAILS REQUIRED] — Name of the University / Institute

[DETAILS REQUIRED] — Department

LABORATORY PROJECT REPORT

StructRender: Engine-Grounded Real-Time Neural Rendering of a Game in Learned Painterly Styles

Submitted by

[DETAILS REQUIRED] — Student name(s)

Register number: [DETAILS REQUIRED]

Course: [DETAILS REQUIRED]

Under the guidance of [DETAILS REQUIRED] — Supervisor name and designation

October 2026

ABSTRACT

Generative world models can produce playable game footage from pixels, but they hold no authoritative game state, so geometry, items and object appearance can drift. **StructRender** takes the opposite approach: a deterministic game engine (Godot 4) keeps sole authority over physics, game logic and entity identity, and a neural network only decides how each frame *looks*. Every frame, the engine exports its own structural buffers (depth, normals, albedo, analytic motion vectors and an entity-identity map), and a compact image-to-image network renders them in a painted style.

Because no paired data exists for artistic styles, a Stable Diffusion 1.5 + depth-ControlNet teacher generates targets offline, accepted only if they pass four pre-registered alignment gates. The deployed renderer is a U-Net whose previous-frame features are warped by the engine's motion field and gated by a disocclusion mask; a 16-dimensional style vector applied through FiLM layers lets one model render two styles. It was trained on 24 recorded play sessions (23,760 frames at 512 × 288; 18/3/3 session split) and deployed through ONNX and TensorRT in a C++ GDExtension.

On the held-out test sessions, the two-style model reaches 35.33 dB PSNR (oil) and 34.79 dB (watercolour), LPIPS 0.0225 and 0.0243, and a sharpness ratio of 0.946 and 0.937 against a 0.90 gate. It flickers about half as much as its own teacher (0.54× and 0.48×). Live in the playable game it runs at 44.6 fps (FP16; 4.53 ms per network step on an RTX 5060 Laptop GPU), with zero fallbacks over 3,000 frames and graceful recovery from injected failures. Negative results are reported as measured: a windowed-transformer renderer never matched the U-Net at parameter parity, and a persistent per-entity appearance memory (EPAM) did not improve output on this data.

Keywords: neural rendering, game engines, image-to-image translation, diffusion teacher, temporal consistency, TensorRT, real-time inference.

TABLE OF CONTENTS

ABSTRACT.....i
LIST OF TABLES.....iii
LIST OF FIGURES.....iv
LIST OF ABBREVIATIONS.....v
CHAPTER 1: INTRODUCTION.....1
1.1 Project Overview.....1
1.2 Problem Statement.....2
1.3 Objectives.....2
1.4 Scope of Work and Social Contribution.....3
CHAPTER 2: LITERATURE REVIEW.....4
2.1 Background.....4
2.1.1 Review of Related Work.....4
2.2 Research Gaps.....6
CHAPTER 3: PROPOSED METHODOLOGY.....7
3.1 Architecture in a Nutshell.....7
3.1.1 Dataset Description.....8
3.1.2 Preprocessing.....9
3.1.3 Encoder Layer (Feature-Extraction and Rendering Network).....10
3.2 Hardware Specifications.....12
3.3 Software Specifications.....13
CHAPTER 4: PERFORMANCE EVALUATION.....14
4.1 Experimental Setup.....14
4.2 Evaluation Metrics.....14
4.3 Results and Analysis.....15
4.3.1 Training Behaviour.....15
4.3.2 Ablation Ladder.....16
4.3.3 The Deployed Two-Style Model.....17
4.3.4 Live Deployment.....18
4.3.5 Assessment of Objectives and Limitations.....19
CHAPTER 5: CONCLUSION AND FUTURE WORK.....20
5.1 Summary of the Contributions.....20
5.2 Future Work.....20
REFERENCES.....21
APPENDICES.....24
Appendix A: Deployed Model Configuration (abridged).....24
Appendix B: Milestone Scorecard.....24
Appendix C: Supplementary Screenshots.....24

LIST OF TABLES

Table 2.1: Comparison of related work....6
Table 2.2: Research gaps and how StructRender addresses them....6
Table 3.1: Dataset summary....8
Table 3.2: Offline teacher configuration....9
Table 3.3: Alignment gates (thresholds fixed before the data)....9
Table 3.4: Parameters of StyleTemporalUNet by block (counted from the model)....12
Table 3.5: Hardware used for all experiments....12
Table 3.6: Software environment (versions read from the environment)....13
Table 4.1: Training and evaluation configuration of the deployed model....14
Table 4.2: Ablation ladder on the sealed seed-scene test split....16
Table 4.3: Game test sessions (2,968 frames per style)....17
Table 4.4: Equivalence chain from evaluator to live game....18
Table 4.5: M10 exit criteria, measured on the playable game....19
Table 4.6: Objectives and the evidence for each....19
Table B.1: Milestones M0–M10 and their verdicts....24

LIST OF FIGURES

Figure 1.1: The playable game rendered natively and by live StructRender (watercolour)……1
Figure 3.1: System architecture: offline data and supervision, and the runtime loop.....7
Figure 3.2: Structural inputs, motion and confidence mask for one test frame.....10
Figure 3.3: StyleTemporalUNet: style-free encoder, temporal fusion, FiLM decoder.....11
Figure 4.1: Training objective and validation L1/PSNR of the deployed model.....16
Figure 4.2: Native input, teacher target and model output for one sealed test frame.....18
Figure 4.3: Live frame rate of the whole loop, and TensorRT latency alone.....19
Figure C.1: Street scene: the original frame (top left) and 16 visual themes.....25
Figure C.2: Swinging scene: the original frame (top left) and 16 visual themes.....25

LIST OF ABBREVIATIONS

CNN	Convolutional neural network
EPAM	Engine-Keyed Persistent Appearance Memory
FiLM	Feature-wise linear modulation
FP16 / FP32 / TF32	16-bit, 32-bit and TensorFloat-32 floating-point formats
fps	Frames per second
GDExtension	Godot's native-code (C++) extension interface
GN	Group normalisation
GPU	Graphics processing unit
KB	The project's knowledge base and Decision Log
L1	Mean absolute error
LPIPS	Learned Perceptual Image Patch Similarity
ONNX	Open Neural Network Exchange
PSNR	Peak signal-to-noise ratio
SD 1.5	Stable Diffusion version 1.5
SSIM	Structural similarity index
TE	Temporal (motion-warp) error
UV	Normalised screen coordinates in [0, 1]

CHAPTER 1: INTRODUCTION

1.1 Project Overview

Changing the *look* of a game, for example rendering it as an oil painting, normally means re-authoring its materials and shaders. Neural rendering offers another route: a network learns to map scene information to an image in a target appearance [1]. Game engines already compute that information every frame as per-pixel depth, normals and albedo, the so-called G-buffer [2]. A second line of work replaces the engine altogether: GameGAN, GameNGen and Genie learn to predict the next frame of a game from earlier frames and player actions [3], [4], [5]. Because such models hold no authoritative state, world facts such as a collected coin or the position of a wall can drift.

StructRender takes the opposite position: **the engine is the single source of truth**, and the network decides appearance only. Every frame, the Godot engine [6] exports its structural buffers and a compact convolutional network [7] renders them in a painted style. The network runs live through ONNX [8], TensorRT [9] and C++. Paired data for painterly styles does not exist, so Stable Diffusion 1.5 [10] with a depth ControlNet [11] is used offline only, to generate training targets. Because the network never touches game state, the style can be switched during play without changing the game (Figure 1.1).

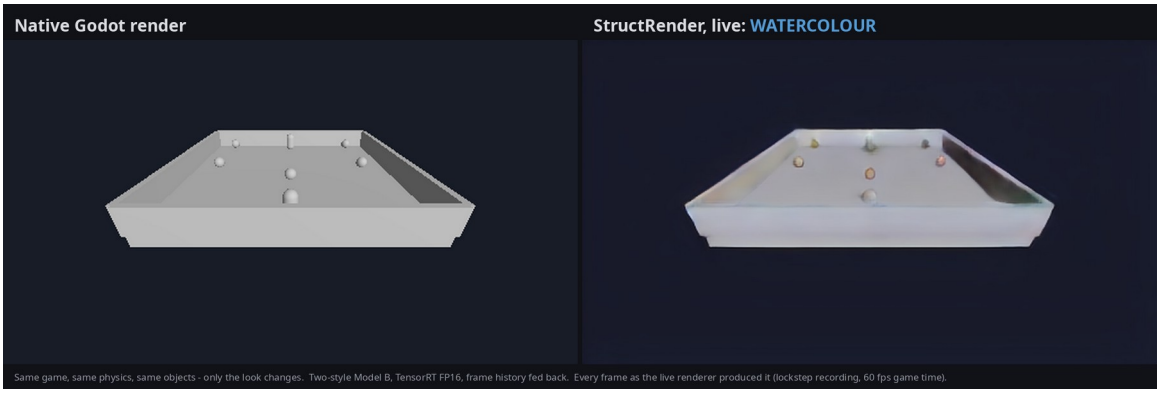


Figure 1.1: The playable game rendered natively and by live StructRender (watercolour)

The work was organised as eleven milestones (M0–M10) whose exit criteria, like every threshold used to judge a result, were fixed before the results existed [12]; negative findings are reported rather than tuned away. Potential applications include stylised rendering for small studios and education, rapid look-development, and research on engine-grounded generative rendering.

1.2 Problem Statement

Problem. Given the structure a game engine already computes for each frame, render that frame in a learned artistic style at interactive frame rates, such that the engine, not the network, determines what exists and where; the output is temporally stable; and the style can be switched while the game runs.

Shortcomings and challenges. Pixel-space world models do not own game state [4], [5]. G-buffer-based enhancement targets photorealism using large real-image datasets [13]. Per-frame stylisation flickers on video [14]. Diffusion models stylise well but need many network evaluations per image [15], [10]. In addition, there is no paired ground truth for painted game frames; a diffusion teacher can invent geometry and is itself temporally unstable; and the live renderer must fit an 8 GB laptop GPU and must provably be the same model that was evaluated.

Measurable outcome. On held-out play sessions: a sharpness ratio of at least 0.90 and colour-histogram travel of at least 0.80 relative to the teacher; a temporal-error reduction beyond a pre-registered noise threshold; about 30 fps for the whole live loop at 512 × 288; standalone network latency of at most about 20 ms; and no gameplay interruption when the renderer fails.

1.3 Objectives

Primary objective. To design, implement and evaluate an engine-grounded neural renderer that re-renders a playable Godot game in learned painterly styles in real time, with the engine retaining authority over all game state.

Secondary objectives.

1. **O1 – Data pipeline.** Export depth, normals, albedo, motion, entity-ID maps and lifecycle events from Godot, and record validated, frame-aligned datasets.
2. **O2 – Supervision.** Generate styled targets with an offline diffusion teacher and accept only targets that pass four pre-defined alignment gates.
3. **O3 – Image quality.** Reach held-out sharpness ≥ 0.90 and colour-histogram travel ≥ 0.80 relative to the teacher.
4. **O4 – Temporal stability.** Reduce temporal error through engine-guided reprojection by more than a pre-registered noise threshold.

5. **O5 – Ablation.** Compare U-Net and transformer backbones, temporal conditioning, entity awareness and appearance memory at parameter parity.
6. **O6 – Style switching.** Render two styles with one model and switch between them during play.
7. **O7 – Deployment.** Run in the game through ONNX, TensorRT and a C++ GDExtension at about 30 fps, with standalone latency ≤ about 20 ms and a native-frame fallback.
8. **O8 – Appearance memory.** Test whether EPAM, a per-entity memory keyed by engine identity, improves output over an identical no-memory model.

1.4 Scope of Work and Social Contribution

Scope. One Godot 4 game (an arena collector with a player, coins and spawned enemies); structural instrumentation; dataset recording and validation; an offline teacher with alignment gates; six renderer variants plus controls; and an ONNX → TensorRT → C++ → Godot runtime, at 512 × 288 with two styles (*oil impasto* and a *bold watercolour*). The network never computes gameplay state; EPAM is session-local; runtime diffusion and mandatory adversarial training are out of scope [12]. All results concern one game and are not evidence about other games or styles.

Contribution and ethics. The project shows that a learned style can be added to a game without letting a generative model alter gameplay, on consumer hardware, and documents its evaluation discipline, including negative results, which makes it useful for teaching. The impact is modest: the styles are mild and the evidence covers one simple game. All data is synthetic and produced by the project; no personal data is collected. The teacher weights are used under their licences (CreativeML OpenRAIL-M [16]; Apache-2.0 [17]). Because the renderer cannot change game state, a renderer error cannot cause a gameplay error.

CHAPTER 2: LITERATURE REVIEW

2.1 Background

G-buffers and U-Nets. Deferred rendering stores per-pixel geometric attributes in separate buffers and shades them later [2]; Godot's Forward+ renderer [6] computes the same attributes, which StructRender re-renders with override shaders and exports. The U-Net [7] is an encoder–decoder CNN whose skip connections carry high-resolution detail to the decoder; it is the standard generator for dense image translation [18]. StructRender's renderer is a four-level U-Net with group normalisation [19], which is independent of batch size and exports cleanly to ONNX.

Style and conditioning. Gatys et al. represented artistic style by correlations of deep CNN features [20]. Dumoulin et al. showed that one network can hold many styles through per-style normalisation parameters [21]. FiLM generalises this as a learned per-channel scale and shift computed from a conditioning vector [22]; StructRender's decoder uses FiLM driven by a learned 16-dimensional style vector.

Diffusion and control. Diffusion models generate images by iterative denoising [15]; latent diffusion made this practical on consumer GPUs [10]. Image-to-image editing starts from a partially noised input, and the noise level (*strength*) trades faithfulness against change [23]. ControlNet conditions a frozen diffusion model on depth, edges or pose through a trainable branch joined by zero-initialised convolutions [11].

Quality and time. SSIM compares local luminance, contrast and structure [24]; LPIPS compares deep features and tracks human judgement [25]. Distortion and perceptual quality are in tension [26], so a regressor trained on pixel error tends to blur; the project therefore pairs pixel metrics with a sharpness measure. Real-time graphics reuses earlier frames by warping them with engine motion vectors [27]; StructRender does this at the network's bottleneck. Window attention (Swin Transformer [28]) underlies the *StructFormer* variant evaluated against the U-Net.

2.1.1 Review of Related Work

The reviewed works fall into four groups; Table 2.1 summarises them. Bibliographic details were checked against Crossref DOI records or arXiv.

Learned rendering from engine buffers. Deep Shading [29] showed that a CNN can map G-buffer attributes to screen-space shading effects at interactive rates. Neural supersampling [27] warps past frames with engine motion vectors and depth to reconstruct high-resolution images in real time. Enhancing Photorealism Enhancement [13] conditions an enhancement network on GTA V's G-buffers and trains it adversarially against real-image datasets such as Cityscapes, achieving stable, geometry-consistent photorealism. These works establish that engine buffers make neural rendering stable; none targets learned artistic styles, for which no real-image domain exists.

Structure-to-image and video translation. pix2pix [18] translates label maps to images with a conditional GAN and a U-Net generator, but needs paired data. vid2vid [30] adds flow-based warping for temporally coherent video, and World-Consistent vid2vid [31] keeps appearance consistent on revisits by projecting earlier output from a reconstructed 3D point cloud, the closest analogue of EPAM's goal. Both rely on estimated flow or geometry and heavy generators.

Temporal consistency. Lai et al. [14] train a post-processing network with short- and long-term temporal losses on optical-flow-warped frames (DAVIS and Videvo data) to remove flicker from per-frame processed video. StructRender's masked temporal loss follows the same principle, but uses exact engine motion and an engine-derived disocclusion mask instead of estimated flow.

Generative game engines. GameGAN [3] learns Pac-Man and VizDoom simulators with an explicit memory module; GameNGen [4] simulates DOOM with a diffusion model conditioned on past frames and actions; Genie [5] learns playable worlds with latent actions from unlabelled Internet 2D-platformer video. All keep game state implicit in pixels or latents, which motivates StructRender's engine-authoritative design. ControlNet [11], finally, supplies the teacher's depth conditioning, and its zero-initialisation idea is reused in StructRender's temporal and FiLM layers so that each new mechanism starts as an exact no-op.

Table 2.1: Comparison of related work

Reference	Year	Methodology	Dataset	Key findings	Limitations (for this project)
Nalbach et al. [29]	2017	CNN on G-buffers → screen-space shading	Rendered 3D scenes	Engine buffers suffice for learned shading	Single frame; photographic goal
Xiao et al. [27]	2020	Motion-vector warping of past frames + CNN	Rendered game-like scenes	Real-time multi-frame reconstruction	Engine look only
Richter et al. [13]	2023	G-buffer-conditioned enhancement, adversarial	GTA V + real datasets	Stable photorealism enhancement	Real-image domain; not artistic
Isola et al. [18]	2017	Conditional GAN, U-Net generator	Paired sets (e.g. Cityscapes)	General structure-to-image translation	Needs paired data; per-frame
Wang et al. [30]	2018	Sequential generator + flow warping	Street-scene video	Temporally coherent synthesis	Estimated flow; heavy model
Mallya et al. [31]	2020	Guidance from 3D point cloud of past output	Street-scene video	Consistency on revisits	Needs reconstructed 3D
Lai et al. [14]	2018	Post-hoc network with temporal losses	DAVIS, Videvo	Task-agnostic consistency	Estimated flow; extra pass
Kim et al. [3]	2020	GAN simulator with memory module	Pac-Man, VizDoom	Learned playable simulation	State implicit; drift
Valevski et al. [4]	2025	Diffusion next-frame model	Agent play of DOOM	Interactive diffusion game	No authoritative state
Bruce et al. [5]	2024	Latent-action foundation world model	Internet platformer video	Playable worlds from video	No engine truth; drift
Zhang et al. [11]	2023	Zero-initialised control branch	Paired condition/image sets	Depth-controlled generation	Slow; no temporal consistency

2.2 Research Gaps

Table 2.2 links each gap to its evidence. G1–G4 follow from the literature; G5–G6 are opportunities identified by the project team.

Table 2.2: Research gaps and how StructRender addresses them

Gap	Evidence	How this project addresses it
G1. Generative game models do not own game state, so world facts can drift	[3], [4], [5]	Engine is the single source of truth; the network renders appearance only
G2. Buffer-conditioned enhancement targets photorealism with real-image data	[13], [29]	Artistic styles supervised by an offline diffusion teacher
G3. Diffusion stylises well but needs many network evaluations per image	[15], [10], [11]	Diffusion offline only; a one-pass CNN runs live
G4. Per-frame stylisation flickers; consistency methods use estimated flow	[14], [30]	Engine motion vectors and an engine-derived disocclusion mask
G5. (Team) No standard acceptance test for aligned stylised targets	Project measurement [12]	Four pre-registered alignment gates; per-frame exclusion
G6. (Team) Appearance memory keyed by engine identity is unexplored	[31]	EPAM tested against a built-in no-memory control

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Architecture in a Nutshell

Figure 3.1 shows the system: an offline part that produces a validated dataset and a trained model, and a runtime loop that runs once per game frame inside Godot.

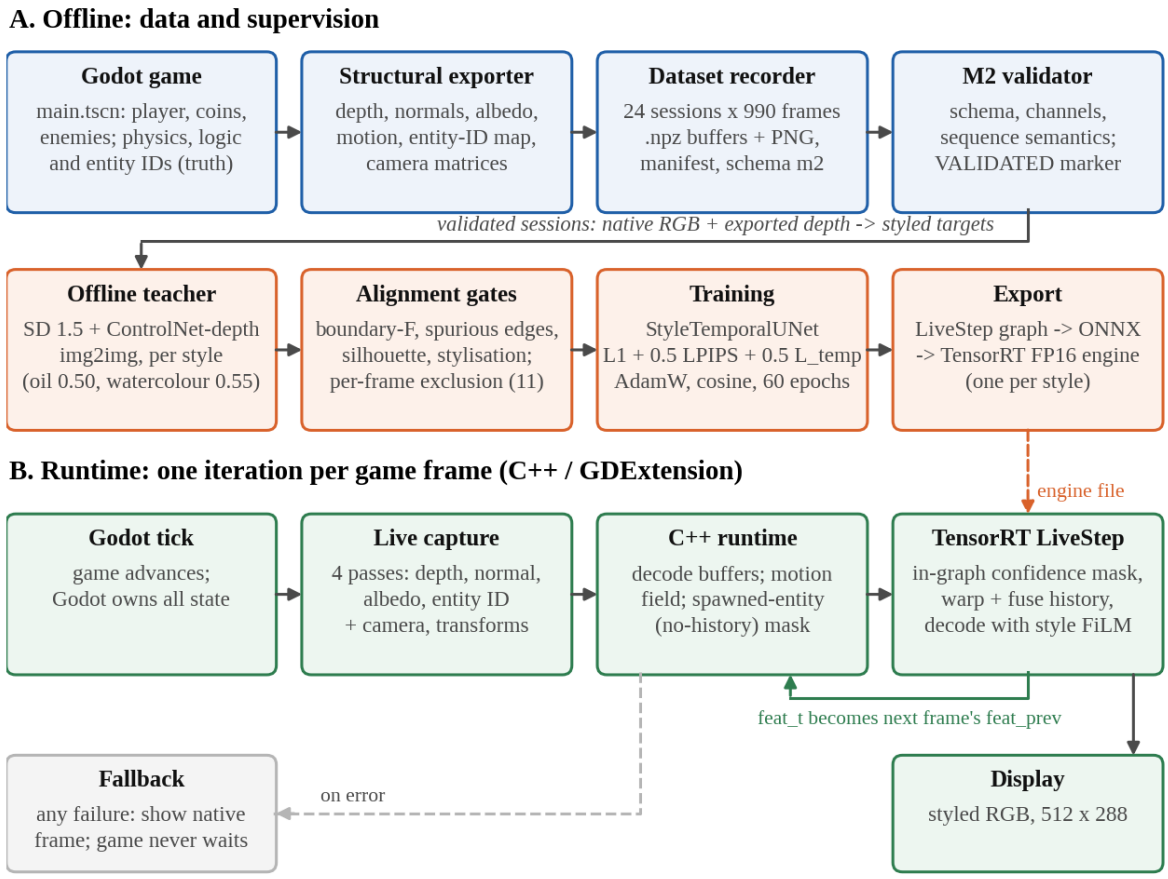


Figure 3.1: System architecture: offline data and supervision, and the runtime loop

- **Game and exporter.** The playable arena collector (`main.tscn`) runs Godot physics at 60 Hz. Override-shader passes re-render each frame as depth, normals, albedo and an entity-ID map; motion vectors are computed analytically from camera and entity transforms.
- **Recorder and validator.** Scripted play drives the game through its own input actions; each frame advances exactly one physics step with the scene paused while passes render, so all buffers show the same instant. The validator rejects sessions with schema, channel or sequence errors.

- **Teacher and gates.** SD 1.5 with a depth ControlNet stylises each native frame, conditioned on the engine's depth; four gates reject targets that lose structure, invent edges, damage silhouettes or fail to stylise.
- **Training and export.** StyleTemporalUNet (Section 3.1.3) learns structure → target with one frame of engine-warped history; one step of it (*LiveStep*), including the confidence mask, is exported to ONNX and compiled to one TensorRT FP16 engine per style on the target laptop.
- **Runtime loop.** Each frame, Godot captures four passes; the C++ GDExtension computes motion and the mask of newly spawned entities, runs the engine with the previous frame's features, displays the result and keeps the new features. Any failure shows the native frame, so the game never waits.

3.1.1 Dataset Description

Both datasets were generated by the project with the same exporter, recorder and validator, so there is no external source, URL or DOI (Table 3.1). The **game dataset**, recorded from the playable game, trained and tested the deployed model. The **seed-scene dataset**, recorded earlier from an arena built in code, was used for the ablation ladder.

Table 3.1: Dataset summary

Property	Game dataset (deployed model)	Seed-scene dataset (ablation)
Sessions	24 recorded, 24 valid	30 recorded, 29 valid
Frames	990 per session; 23,760 total	990 per session; 28,710 total
Resolution	512 × 288	512 × 288
Split (sessions)	18 / 3 / 3 (train / val / test)	21 / 4 / 4
Split (frames)	17,820 / 2,970 / 2,970	20,790 / 3,960 / 3,960
Targets (styles)	oil_impasto 0.50; watercolor_bold 0.55	oil_impasto 0.50
Excluded by gates	11 frames (0.05 %)	whole-session rule
Per-frame record	native RGB (PNG); depth, normals, albedo, motion (float16); entity ID (int32); camera matrices; lifecycle events	same
Size / licence	2.9 GB structural data; project-generated	same

The task is dense regression from structure to a styled RGB image, so there are no classes; a frame's *label* is the teacher image for the selected style, stored in a per-style sidecar dataset. No manual annotation is involved. **Limitations:** the game uses flat, untextured grey geometry, so the targets are mild; only a few entity types occur; routes are scripted rather than human; and

the 11 excluded frames are the hardest-to-align views, which are therefore slightly under-represented [12].

3.1.2 Preprocessing

(a) Validation. Sessions with schema errors, partially dropped channels, non-finite values or inconsistent lifecycle sequences are rejected; one seed-scene session was rejected for non-finite motion before reaching training.

(b) Teacher targets. For each validated frame and style, SD 1.5 img2img with a depth ControlNet generates the target, conditioned on the exported engine depth (Table 3.2). Generation is seed-pinned and cached by content hash.

Table 3.2: Offline teacher configuration

Parameter	oil_impasto	watercolor_bold_s055
Models	SD 1.5 [16] + ControlNet depth [17]	same
Steps / guidance / ControlNet scale	24 / 7.5 / 1.6	24 / 7.5 / 1.6
img2img strength / seed	0.50 / 20260906	0.55 / 20260930
Prompt (abridged)	thick oil painting, heavy impasto, ochre and deep blue palette	bold watercolour on cream paper, saturated washes, colour blooms

(c) Alignment gates. Each target is scored against the engine's own structure with thresholds fixed before the game data existed (Table 3.3). Gate 4 exists because gates 1–3 are passed trivially by a teacher that changes nothing. Style strength was chosen by a pre-registered rule (strongest setting with zero failures on a 45-frame probe, then a full 990-frame session); the original watercolour failed on the game's flat grey surfaces and a bolder prompt passed at 0.55 [12].

Table 3.3: Alignment gates (thresholds fixed before the data)

Gate	Measures	Threshold
1. Boundary F-measure	structural edges preserved (2 px tolerance)	≥ 0.60
2. Spurious-edge ratio	strong edges with no structural support (invented geometry)	≤ 0.35
3. Silhouette support	each entity of ≥ 200 visible px keeps its outline	≥ 0.60
4. Stylisation	colour-histogram L1 distance from the native frame	≥ 0.15

(d) Exclusion and leakage prevention. A frame failing any gate in either style is excluded from both, with every temporal sample touching it (11 of 23,760 frames). The split is by **whole**

session, fixed by position before any target or model existed; the test split was scored once, at the end.

(e) Input encoding. The input has 7 channels: inverse depth, normals (3) and albedo (3). Depth *d* in metres is bounded, with background (d = +∞) mapped to 0:

$$x_{depth} = 1 / (1 + d) \tag{3.1}$$

Motion and entity IDs are deliberately **not** input channels: motion is used for warping, and IDs are addresses, since a numeric ID channel would make a recycled ID look like the entity it replaced [12]. No augmentation is used, because it would break the exact correspondence between structure, motion and target.

(f) Confidence mask. A training sample is three consecutive frames (t−2, t−1, t). A per-pixel mask M_t decides which pixels may trust history. With $W(\cdot, V_t)$ the backward warp by the engine motion field, D depth and N normals:

$$M_t = \exp(-\alpha\,|D_t - W(D_{t-1}, V_t)| - \beta\,(1 - N_t \cdot W(N_{t-1}, V_t))) \cdot H_t \tag{3.2}$$

where α = ln 2 / 0.175 m and β = ln 2 / (1 − cos 30°), and $H_t \in \{0, 1\}$ zeroes pixels with no valid history: frame 0, off-screen motion, background, and newly spawned entities. The 0.175 m scale was calibrated on the data as the best separation (Youden index) between depth steps within surfaces and across silhouettes. Figure 3.2 shows these inputs for a sealed test frame.

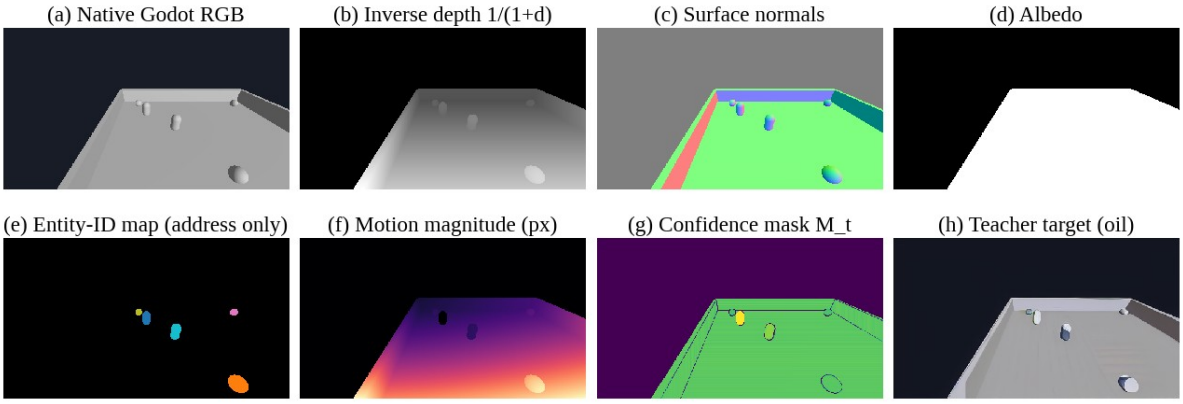


Figure 3.2: Structural inputs, motion and confidence mask for one test frame

3.1.3 Encoder Layer (Feature-Extraction and Rendering Network)

Heading adapted to the project: the network is a convolutional encoder–decoder whose encoder is the feature extractor. The deployed **StyleTemporalUNet** (Figure 3.3, Table 3.4) has 8,394,915 parameters.

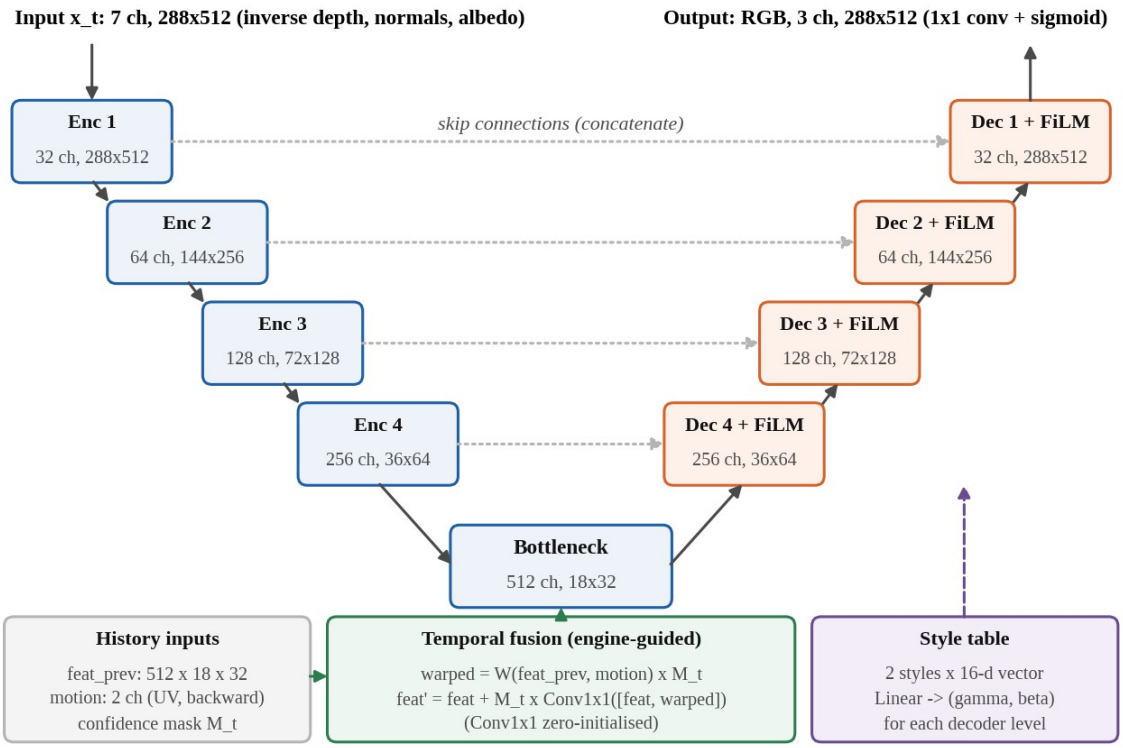


Figure 3.3: StyleTemporalUNet: style-free encoder, temporal fusion, FiLM decoder

Encoder. Four levels, each of two 3 × 3 convolutions with group normalisation (8 groups) [19] and ReLU followed by 2 × 2 max pooling, give 32, 64, 128 and 256 channels at 288 × 512 down to 36 × 64; a bottleneck produces 512 channels at 18 × 32. The encoder reads only structure, never the style, so its features are the same whichever style is shown and can serve as history across a live style switch.

Temporal fusion. The previous frame's bottleneck features are warped by the engine motion, masked, and fused through a zero-initialised 1 × 1 convolution, so an untrained model equals the single-frame U-Net and history is ignored wherever $M_t = 0$:

$$F'_t = F_t + M_t \odot \mathrm{Conv}_{1\times1}(\,[\,F_t\,,\, M_t \odot W(F_{t-1}, V_t)\,]\,) \tag{3.3}$$

Decoder with FiLM. Each level upsamples by 2, concatenates the encoder skip, applies two convolution blocks, and is modulated by FiLM [22] from the selected style's learned 16-dimensional vector s; zero-initialised projections make an untrained two-style model exactly the single-style one. A 1 × 1 convolution and sigmoid output RGB in [0, 1].

$$h' = h \odot (1 + \gamma_s) + \beta_s\,, \quad [\,\gamma_s\,;\,\beta_s\,] = A_l\, s + b_l \tag{3.4}$$

Table 3.4: Parameters of StyleTemporalUNet by block (counted from the model)

Block	Output (channels × H × W)	Parameters
Encoder, 4 levels	32 × 288 × 512 → 256 × 36 × 64	1,175,328
Bottleneck	512 × 18 × 32	3,542,016
Temporal fusion (1 × 1 conv)	512 × 18 × 32	524,800
Decoder, 4 levels	256 × 36 × 64 → 32 × 288 × 512	3,136,320
FiLM projections + style table	per decoder level	16,320 + 32
Head (1 × 1 conv + sigmoid)	3 × 288 × 512	99
Total		**8,394,915**

Training objective. With teacher target I_t, prediction $\hat{I}_t$ and pixel p, the same weights are used for every model in the project; the temporal term uses the prediction on both sides and the same mask as the evaluation metric:

$$L = \| \hat{I}_t - I_t \|_1 + 0.5 \cdot \mathrm{LPIPS}(\hat{I}_t, I_t) + 0.5 \cdot \Sigma_p M_t \, | \hat{I}_t - W(\hat{I}_{t-1}, V_t) | \, / \, \Sigma_p M_t \qquad (3.5)$$

Comparison models. For the ablation, Model A is the same U-Net without history; StructFormer (Models C–F) uses a convolutional stem, window-attention encoder and decoder [28] and a convolutional head; E adds entity-masked cross-attention and F adds the EPAM memory. All rungs were trained at parameter parity (0.9–1.1×) with the same data, seed, objective and selection rule [12].

3.2 Hardware Specifications

All recording, teacher generation, training, evaluation and live deployment ran on one laptop (Table 3.5); the project's logs record no cloud run behind any reported result. TensorRT engines were built on this GPU, as engines are specific to the GPU and version they are built with [9].

Table 3.5: Hardware used for all experiments

Component	Specification
Machine	Laptop — make and model [DETAILS REQUIRED]
CPU	AMD Ryzen 9 8940HX, 16 cores / 32 threads
System memory	30 GiB visible to the operating system
GPU	NVIDIA GeForce RTX 5060 Laptop GPU, 8 GB (Blackwell, compute capability 12.0), driver 610.57.04
Storage	2 × 1 TB NVMe SSD
Operating system	Omarchy (Arch Linux based), Linux kernel 7.2.5

The CPU runs Godot, data loading (8 workers) and the C++ motion computation; the GPU runs teacher generation, bfloat16 training, evaluation and TensorRT inference. The 8 GB of GPU memory, shared with the desktop display, was the binding constraint: training used batch 4 with gradient accumulation 2 (effective batch 8).

3.3 Software Specifications

Table 3.6: Software environment (versions read from the environment)

Category	Software and version	Purpose
Game engine	Godot 4.7.2, Forward+ [6]	Game, structural export, live loop
Native extension	C++17, godot-cpp [32], GCC 16.2.1, CMake 4.4.4	GDExtension and C++ runtime
Deep learning	Python 3.10.21; PyTorch 2.13.0 (CUDA 13.2) [33]; torchvision 0.28.0	Models, training, evaluation
Inference	TensorRT 11.2.1.2 [9]; ONNX 1.22.0 [8]; ONNX Runtime 1.23.2	Export and runtime engines
Teacher	Diffusers 0.40.0 [34]; Transformers 5.16.1	Target generation
Metrics and tools	lpips 0.1.4 [25]; NumPy 2.2.6; matplotlib 3.10.9; pytest 9.1.1	Metrics, figures, tests

Godot scripts record the dataset; Python reads it for teacher generation, training and evaluation; a checkpoint is exported once to ONNX and compiled by TensorRT; the C++ GDExtension loads the engines and is called from GDScript every frame. Python and C++ share one TensorRT build, so engines and numerical checks are shared.

CHAPTER 4: PERFORMANCE EVALUATION

4.1 Experimental Setup

The environment, data and preprocessing are given in Chapter 3. Table 4.1 lists the configuration of the deployed model (`configs/game/model_b_two_style.json`); the ablation rungs used the same optimiser, schedule, objective and selection rule on the seed-scene dataset.

Table 4.1: Training and evaluation configuration of the deployed model

Item	Setting
Data	18 training sessions, both styles: 35,526 training samples; 5,936 validation frames
Objective	1 · L1 + 0.5 · LPIPS (AlexNet) + 0.5 · temporal term — Eq. (3.5)
Optimiser / schedule	AdamW [35], learning rate 2×10^{-4}, weight decay 10^{-4}; cosine annealing [36]
Batch / precision	4 × gradient accumulation 2 (effective 8); bfloat16 mixed precision
Epochs / stopping	60 maximum, validation every 2; early-stopping patience 6 evaluations (not triggered)
Selection / seed	lowest validation L1 over both styles; seed 20260905
Wall time / best	29.2 h; best epoch 49 (validation L1 0.007435)
Test protocol	3 sealed sessions, 2,968 frames per style, each with its real previous frame; flicker on 2,451 moving pairs

Baselines. The ablation ladder compares A (U-Net), B (A + temporal), C (StructFormer), D (C + temporal), E (C + entity masks), the control N (C + raw ID channel), and F/F0 (E with and without EPAM). On the game, the deployed model is compared with an oil-only Model B and with the unstyled native frame (the *do-nothing* anchor).

Reproducibility. Thresholds were written into evaluator code or the Decision Log before results existed; seeds, configurations and JSON reports are under version control; weights are published as GitHub releases. Seed variance measured on the seed scene (three seeds of Models C and E) sets the noise thresholds; game models were trained once, so small differences between them are not claimed. The automated test suite (548 tests) passed in full when re-run on 23 October 2026.

4.2 Evaluation Metrics

The task is image regression with temporal and real-time requirements, so classification metrics do not apply. Below, $\hat{I}$ is the prediction and I the target in [0, 1], and N the number of pixel values.

L1 and PSNR. Mean absolute error, and peak signal-to-noise ratio in dB (higher is better); both reward blur when targets are uncertain [26].

$$L1 = (1/N)\,\Sigma_i \,|\, \hat{I}_i - I_i \,|\,, \quad PSNR = 10 \cdot \log_{10}(\,1 / MSE\,) \tag{4.1}$$

SSIM [24] compares local means μ, variances σ^2 and covariance σ_{xy} (11 × 11 Gaussian windows; 1 = identical). **LPIPS** [25] is a distance between deep AlexNet features (lower is better); being part of the objective, it is never quoted alone.

$$SSIM = (2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2) \,/\, ((\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)) \tag{4.2}$$

Sharpness ratio. High-pass energy of the prediction relative to the target (squared Sobel gradients of luma Y, summed over the test set). A value of 0.90 corresponds to roughly a 1-pixel Gaussian blur of the target. It is a blur guard, not proof of quality.

$$S = \Sigma \,\|\, \nabla Y(\hat{I}) \,\|^2 \;/\; \Sigma \,\|\, \nabla Y(I) \,\|^2 \tag{4.3}$$

Histogram travel (style acquisition). With h the 32-bin per-channel colour histogram and I_{nat} the native frame, the ratio of how far the model moves the palette relative to the teacher (gate ≥ 0.80).

$$\text{travel ratio} = \|\, h(\hat{I}) - h(I_{nat}) \,\|_1 \;/\; \|\, h(I) - h(I_{nat}) \,\|_1 \tag{4.4}$$

Temporal error (flicker). Mean difference between a frame and the previous frame warped by engine motion, on confident pixels, over pairs with at least 1 pixel of median motion; lower is steadier. Since a frozen output scores zero, it is always reported with sharpness.

$$TE = \Sigma_p \, M_t \,|\, I_t - W(I_{t-1}, V_t) \,|\; /\; \Sigma_p \, M_t \tag{4.5}$$

Noise threshold. A difference between two single runs is claimed only if it exceeds $2\sqrt{2} \cdot sd_{rel}$, where sd_{rel} is the relative seed-to-seed standard deviation. **Runtime.** Frames per second = 1000 / median loop time (ms) for the whole live loop; engine latency is the median of 500 standalone TensorRT runs; the fallback rate is the fraction of styled frames that showed the native image.

4.3 Results and Analysis

4.3.1 Training Behaviour

Figure 4.1 shows the training of the deployed model. Validation L1 reaches its minimum of 0.007435 at epoch 49 and then flattens. The temporal term rises during training as the initially

smooth (hence stable) output acquires the teacher's texture, which is why temporal error is always reported with sharpness.

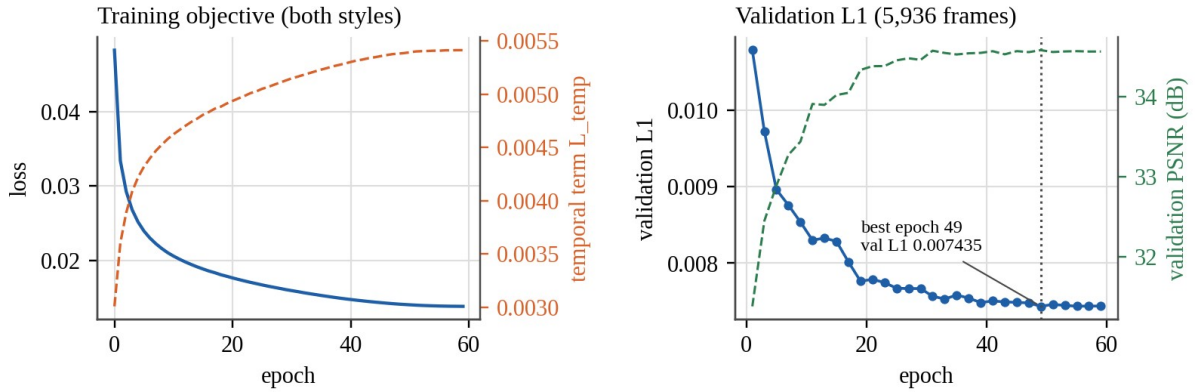


Figure 4.1: Training objective and validation L1/PSNR of the deployed model

4.3.2 Ablation Ladder

Table 4.2 reports every rung on the same 3,960 sealed test frames of the seed scene, scored by one evaluation script; temporal error is measured on 501 moving pairs.

Table 4.2: Ablation ladder on the sealed seed-scene test split

Rung	PSNR (dB)	SSIM	LPIPS	Sharpness	Temporal error	Parameters
A U-Net	**34.04**	**0.9747**	**0.0459**	**0.912**	**0.003175**	**7.85 M**
B U-Net + temporal	33.64	0.9734	0.0489	0.897	0.002692 (−15.2 %)	8.38 M
C StructFormer	33.86	0.9728	0.0466	0.853	0.004191	7.78 M
D StructFormer + temporal	33.56	0.9727	0.0485	0.858	0.003741 (−10.7 %)	8.24 M
E StructFormer + entity	33.79	0.9726	0.0463	0.884	0.004172	7.05 M
N raw-ID control	33.86	0.9732	0.0459	0.880	–	7.78 M
Teacher targets / native engine	–	–	–	–	0.006265 / 0.000241	–

The plain U-Net (A) is best on every image-quality metric and is the only rung above the 0.90 sharpness gate. StructFormer's sharpness was 0.833, 0.886 and 0.851 at 644, 10,890 and 20,790 training frames, against a pre-recorded projection of 0.898: more data did not close the gap, so at this scale the CNN's inductive bias suits the task better. **Temporal conditioning cuts flicker on both backbones** by 15.2 % (B vs A) and 10.7 % (D vs C), well beyond the pre-registered threshold of 3.85 % ($2\sqrt{2} \times 1.36$ %), at a cost of 1.8–3.9 % in per-frame L1; since flicker is the more visible defect live, Model B was chosen for the game. **Entity awareness** sharpens StructFormer (+0.015 over C across three seeds, 2.5σ), but the naive raw-ID control N recovers about 87 % of that gain: with two or three stable IDs in this scene, exposing the IDs is enough.

EPAM (O8) is a negative result. Model F (memory on) and its twin F0 (memory off) share code, data, schedule and selection rule. On 3,912 test clips, F − F0 sharpness is −0.0127 (0.8829 vs 0.8956), inside the pre-registered threshold of 0.0206 and in the wrong direction. Three independent measurements agree that this data gives memory nothing to recover, most likely because the diffusion teacher re-invents each object's look every frame. The memory's lifecycle rules (retrieval after leaving view, no leakage after destruction, clearing on style change) are met and covered by 17 tests.

4.3.3 The Deployed Two-Style Model

Table 4.3: Game test sessions (2,968 frames per style)

Model, style	PSNR (dB)	SSIM	LPIPS	Sharpness	Travel ratio	Flicker vs teacher
Two-style B, oil	35.33	0.9807	0.0225	0.946	0.995	0.54×
Two-style B, watercolour	34.79	0.9786	0.0243	0.937	0.987	0.48×
Oil-only B, oil (reference)	35.43	0.9814	0.0216	0.946	0.980	0.54×
Do-nothing (native frame), oil / watercolour	28.06 / 26.96	–	–	–	–	–

Table 4.3 shows that both styles clear the sharpness and travel gates and beat doing nothing by 7.3–7.8 dB. Sharing one model between two styles costs oil 0.10 dB, which is not claimed as significant with one run per model. The renderer flickers about half as much as its own teacher but still about 57× the native engine. The style switch is real, not a blend: on a sampled test session each style's output is about 4× closer to its own targets than to the other style's, reproducing 93 % of the difference between the two teachers. Figure 4.2 shows a sealed test frame rendered through exactly the evaluated path: geometry is the engine's in every column; only appearance changes. The styles are mild because the game is grey and plainly lit, and stronger settings fail the gates.

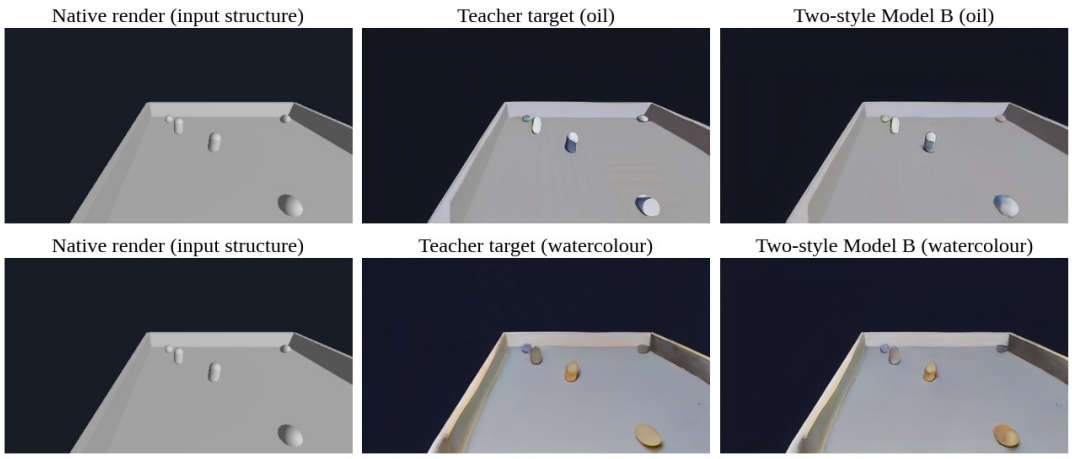


Figure 4.2: Native input, teacher target and model output for one sealed test frame

Value of history. Run as a cold start every frame, the same model drops from 35.33 to 33.58 dB (oil) and from 34.79 to 32.82 dB (watercolour), so the live runtime feeds history back every frame.

4.3.4 Live Deployment

To ensure the game shows the evaluated model, each link from evaluator to screen was checked with identical inputs (Table 4.4).

Table 4.4: Equivalence chain from evaluator to live game

Link	Maximum difference
Exported LiveStep graph vs Python evaluator (RGB)	2.4×10^{-6}
TensorRT strict FP32 vs PyTorch (RGB / fed-back features)	1.6×10^{-6} / 7.3×10^{-6}
C++ motion vs engine GDScript motion (120 lockstep frames)	7.5×10^{-8} UV
Live game frame vs evaluated model	1/255 (rounding); 0 pixels above
Enemy spawned mid-game: no-history pixels, C++ vs engine ID map	998 = 998 pixels

The FP16 engine passed a tolerance fixed before measurement (mean $|\Delta\text{PSNR}| < 0.05$ dB, mean pixel change < 1/255) with 0.044 / 0.039 dB and 0.0004; its few larger pixel differences lie on one-pixel object outlines. Figure 4.3 and Table 4.5 report the live measurements on the playable game.

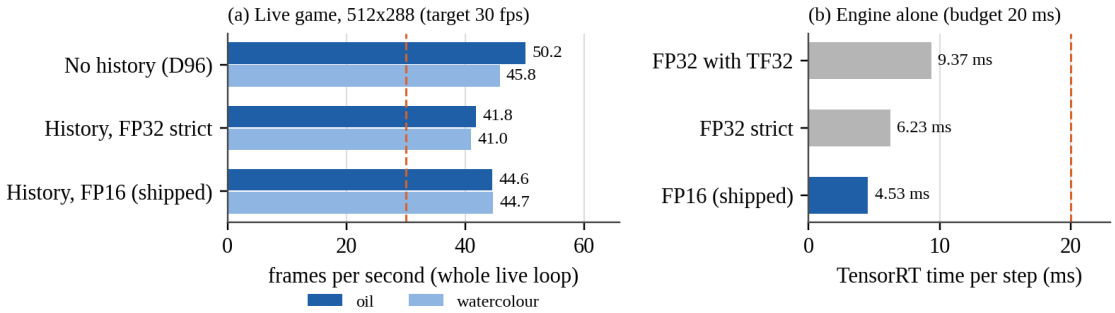


Figure 4.3: Live frame rate of the whole loop, and TensorRT latency alone

Table 4.5: M10 exit criteria, measured on the playable game

Criterion (KB target)	Result	Verdict
Stable live loop	3,000 frames, 0 fallbacks; median 22.4 ms (first half) vs 22.3 ms (second half); p99 25.2 ms	Met
About 30 fps at 512 × 288	44.6 fps oil, 44.7 fps watercolour (FP16)	Met
TensorRT alone ≤ about 20 ms	4.53 ms FP16; 6.23 ms strict FP32; 9.37 ms with TF32	Met
Failure does not break gameplay	8 of 8 forced failures fell back to the native frame; play continued; next frame rendered	Met

FP16 is 27 % faster than strict FP32, so it ships. In a manual play session on 23 October 2026 (free play, desktop applications open), 2,059 styled frames ran at 40.1 fps (oil) and 38.0 fps (watercolour) with **0 fallbacks**.

4.3.5 Assessment of Objectives and Limitations

Table 4.6: Objectives and the evidence for each

Objective	Status	Evidence
O1 Data pipeline	Achieved	24/24 game and 29/30 seed sessions validated; corrupt session rejected
O2 Gated supervision	Achieved	Two gate-passing styles; 11 of 23,760 frames excluded by rule
O3 Image quality	Achieved	Sharpness 0.946 / 0.937; travel 0.995 / 0.987
O4 Temporal stability	Achieved	−15.2 % and −10.7 % vs 3.85 % threshold; 0.54× / 0.48× teacher flicker
O5 Ablation	Achieved	All rungs at parity on one scoring path; U-Net best
O6 Style switching	Achieved	One model, two styles; switch keeps history
O7 Deployment	Achieved	44.6 fps; 4.53 ms; 0 fallbacks in 3,000 frames
O8 EPAM improves output	Not achieved	F − F0 = −0.0127 sharpness, inside noise

Limitations. The evidence covers one simple game and two mild styles; teacher flicker bounds temporal quality; game models were trained once; and style strength is limited by the gates. A move to 768 × 432 was stopped before the bulk run because the teacher's look changes with resolution: the best re-tuned settings were no closer to the committed look than the unstyled frame (LPIPS 0.094 / 0.100 vs 0.099 / 0.117).

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Summary of the Contributions

The project set out to add a learned artistic style to a playable game without letting a neural network decide anything about the game. It delivered a complete, verified pipeline, from engine instrumentation and a 23,760-frame validated dataset to a gated diffusion teacher, a temporally conditioned two-style U-Net, and an ONNX → TensorRT → C++ → Godot runtime. Its main contributions are:

1. **An engine-grounded design** in which structure is the input, engine motion drives warping, and entity identity addresses masks and memory; none of them becomes a source of gameplay truth.
2. **Gated supervision from a diffusion teacher**, including a stylisation gate that rejects a teacher that does nothing, and an explicit per-frame exclusion rule.
3. **A style-free encoder with FiLM decoding**, so one 8.39 M-parameter model renders two styles and keeps its temporal memory across a live switch.
4. **A verified live deployment** at 44.6 fps (4.53 ms per FP16 step), with a link-by-link proof that the live frame equals the evaluated model's frame within rounding, and **pre-registered, honest evaluation**, including two negative results (StructFormer and EPAM).

5.2 Future Work

- **A temporally consistent teacher** (video-consistent diffusion or authored per-entity textures), which bounds both stability and EPAM's usefulness.
- **Re-testing EPAM** in scenes with many entities, occlusions and recycled IDs, as a new pre-registered experiment with repeated seeds.
- **Higher resolution with a matched look**, for example super-resolution from 512 × 288 targets, or new styles designed at 768 × 432.
- **Broader evaluation:** more games and textured scenes, human-play data, seed variance on the game, optional adversarial sharpening, and INT8/FP8 inference.

REFERENCES

[1] A. Tewari et al., "State of the art on neural rendering," Comput. Graph. Forum, vol. 39, no. 2, pp. 701–727, 2020, doi: 10.1111/cgf.14022.

[2] T. Saito and T. Takahashi, "Comprehensible rendering of 3-D shapes," ACM SIGGRAPH Comput. Graph., vol. 24, no. 4, pp. 197–206, 1990, doi: 10.1145/97880.97901.

[3] S. W. Kim, Y. Zhou, J. Philion, A. Torralba, and S. Fidler, "Learning to simulate dynamic environments with GameGAN," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2020, pp. 1228–1237, doi: 10.1109/CVPR42600.2020.00131.

[4] D. Valevski, Y. Leviathan, M. Arar, and S. Fruchter, "Diffusion models are real-time game engines," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025, arXiv:2408.14837.

[5] J. Bruce et al., "Genie: Generative interactive environments," arXiv preprint arXiv:2402.15391, 2024.

[6] Godot Engine contributors, "Godot Engine documentation." [Online]. Available: https://docs.godotengine.org/en/stable/ (accessed Oct. 23, 2026).

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[8] ONNX Project, "Open Neural Network Exchange (ONNX)." [Online]. Available: https://onnx.ai/ (accessed Oct. 23, 2026).

[9] NVIDIA Corporation, "NVIDIA TensorRT documentation." [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/latest/ (accessed Oct. 23, 2026).

[10] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2022, pp. 10674–10685, doi: 10.1109/CVPR52688.2022.01042.

[11] L. Zhang, A. Rao, and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2023, pp. 3813–3824, doi: 10.1109/ICCV51070.2023.00355.

[12] StructRender project, "StructRender: System architecture and knowledge base, including the Decision Log (Decisions 1–102)," unpublished technical document, docs/StructRender_KB.md, Oct. 2026.

[13] S. R. Richter, H. A. Alhaija, and V. Koltun, "Enhancing photorealism enhancement," IEEE Trans. Pattern Anal. Mach. Intell., vol. 45, no. 2, pp. 1700–1715, 2023, doi: 10.1109/TPAMI.2022.3166687.

[14] W.-S. Lai, J.-B. Huang, O. Wang, E. Shechtman, E. Yumer, and M.-H. Yang, "Learning blind video temporal consistency," in Computer Vision – ECCV 2018 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2018, pp. 179–195, doi: 10.1007/978-3-030-01267-0_11.

[15] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Advances in Neural Information Processing Systems (NeurIPS), 2020, arXiv:2006.11239.

[16] Hugging Face model repository "stable-diffusion-v1-5/stable-diffusion-v1-5," revision 451f4fe1. [Online]. Available: https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-v1-5 (accessed Oct. 23, 2026).

[17] Hugging Face model repository "lllyasviel/sd-controlnet-depth," revision 35e42a3e. [Online]. Available: https://huggingface.co/lllyasviel/sd-controlnet-depth (accessed Oct. 23, 2026).

[18] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 5967–5976, doi: 10.1109/CVPR.2017.632.

[19] Y. Wu and K. He, "Group normalization," in Computer Vision – ECCV 2018 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2018, pp. 3–19, doi: 10.1007/978-3-030-01261-8_1.

[20] L. A. Gatys, A. S. Ecker, and M. Bethge, "Image style transfer using convolutional neural networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 2414–2423, doi: 10.1109/CVPR.2016.265.

[21] V. Dumoulin, J. Shlens, and M. Kudlur, "A learned representation for artistic style," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017, arXiv:1610.07629.

[22] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in Proc. AAAI Conf. Artif. Intell., vol. 32, no. 1, 2018, doi: 10.1609/aaai.v32i1.11671.

[23] C. Meng, Y. He, Y. Song, J. Song, J. Wu, J.-Y. Zhu, and S. Ermon, "SDEdit: Guided image synthesis and editing with stochastic differential equations," in Proc. Int. Conf. Learn. Represent. (ICLR), 2022, arXiv:2108.01073.

[24] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004, doi: 10.1109/TIP.2003.819861.

[25] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 586–595, doi: 10.1109/CVPR.2018.00068.

[26] Y. Blau and T. Michaeli, "The perception-distortion tradeoff," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 6228–6237, doi: 10.1109/CVPR.2018.00652.

[27] L. Xiao, S. Nouri, M. Chapman, A. Fix, D. Lanman, and A. Kaplanyan, "Neural supersampling for real-time rendering," ACM Trans. Graph., vol. 39, no. 4, 2020, doi: 10.1145/3386569.3392376.

[28] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision transformer using shifted windows," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2021, pp. 9992–10002, doi: 10.1109/ICCV48922.2021.00986.

[29] O. Nalbach, E. Arabadzhiyska, D. Mehta, H.-P. Seidel, and T. Ritschel, "Deep shading: Convolutional neural networks for screen space shading," Comput. Graph. Forum, vol. 36, no. 4, pp. 65–78, 2017, doi: 10.1111/cgf.13225.

[30] T.-C. Wang, M.-Y. Liu, J.-Y. Zhu, G. Liu, A. Tao, J. Kautz, and B. Catanzaro, "Video-to-video synthesis," in Advances in Neural Information Processing Systems (NeurIPS), 2018, arXiv:1808.06601.

[31] A. Mallya, T.-C. Wang, K. Sapra, and M.-Y. Liu, "World-consistent video-to-video synthesis," in Computer Vision – ECCV 2020 (Lecture Notes in Computer Science). Cham, Switzerland: Springer, 2020, pp. 359–378, doi: 10.1007/978-3-030-58598-3_22.

[32] Godot Engine contributors, "godot-cpp: C++ bindings for the Godot script API (GDExtension)," GitHub repository. [Online]. Available: https://github.com/godotengine/godot-cpp (accessed Oct. 23, 2026).

[33] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019, arXiv:1912.01703.

[34] Hugging Face, "Diffusers: State-of-the-art diffusion models," GitHub repository. [Online]. Available: https://github.com/huggingface/diffusers (accessed Oct. 23, 2026).

[35] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019, arXiv:1711.05101.

[36] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in Proc. Int. Conf. Learn. Represent. (ICLR), 2017, arXiv:1608.03983.

APPENDICES

Appendix A: Deployed Model Configuration (abridged)

```
{ "data":  { "styles": ["oil_impasto", "watercolor_bold_s055"],
             "split_file": "configs/data/game_split_v1.json",
             "frame_exclusions":
                 "configs/data/game_frame_exclusions_v1.json" },
  "model": { "in_channels": 7, "out_channels": 3, "base_channels": 32,
             "image_size": [288, 512], "arch": "style_temporal_unet" },
  "train": { "seed": 20260905, "epochs": 60, "lr": 0.0002,
             "weight_decay": 0.0001, "amp_dtype": "bfloat16",
             "batch_size": 4, "accum_steps": 2, "eval_every_epochs": 2,
             "early_stopping": { "patience_evals": 6,
                                 "min_rel_delta": 0.001 } },
  "loss":  { "l1": 1.0, "perceptual": 0.5, "perceptual_net": "alex",
             "temporal": 0.5 } }
```

Evidence for every number in this report is in the repository: `eval/reports/` (JSON reports), `eval/logs/` (logs) and `docs/StructRender_KB.md` (Decision Log).

Appendix B: Milestone Scorecard

Table B.1: Milestones M0–M10 and their verdicts

Milestone	Verdict
M0 Repository and minimal Godot game	Complete
M1 Structural export	Complete
M2 Dataset recorder and validator	Complete
M3 Offline teacher targets and alignment gates	Complete
M4 CNN / U-Net baseline	Passes at every dataset size
M5 StructFormer baseline	Fails: below the U-Net at all three data sizes
M6 Temporal reprojection	Passes; replicated on new data
M7 Structured entity awareness	Complete
M8 EPAM	Criteria 1–3 met; criterion 4 (improves output) not met
M9 Full evaluation (ablation ladder)	Complete
M10 ONNX → TensorRT → C++ → Godot	Passes all exit criteria

Appendix C: Supplementary Screenshots

Figures C.1 and C.2 show screenshots of https://structrender.vercel.app/, a separate project: a Spider-Man web-swinging game for the browser (Three.js), and an unofficial fan project not affiliated with Marvel, Disney, Sony or Insomniac Games. Its 16 themes are drawn by its own rendering pipeline, not by the StructRender model of Chapter 4. The originals are in `docs/screenshots/`.

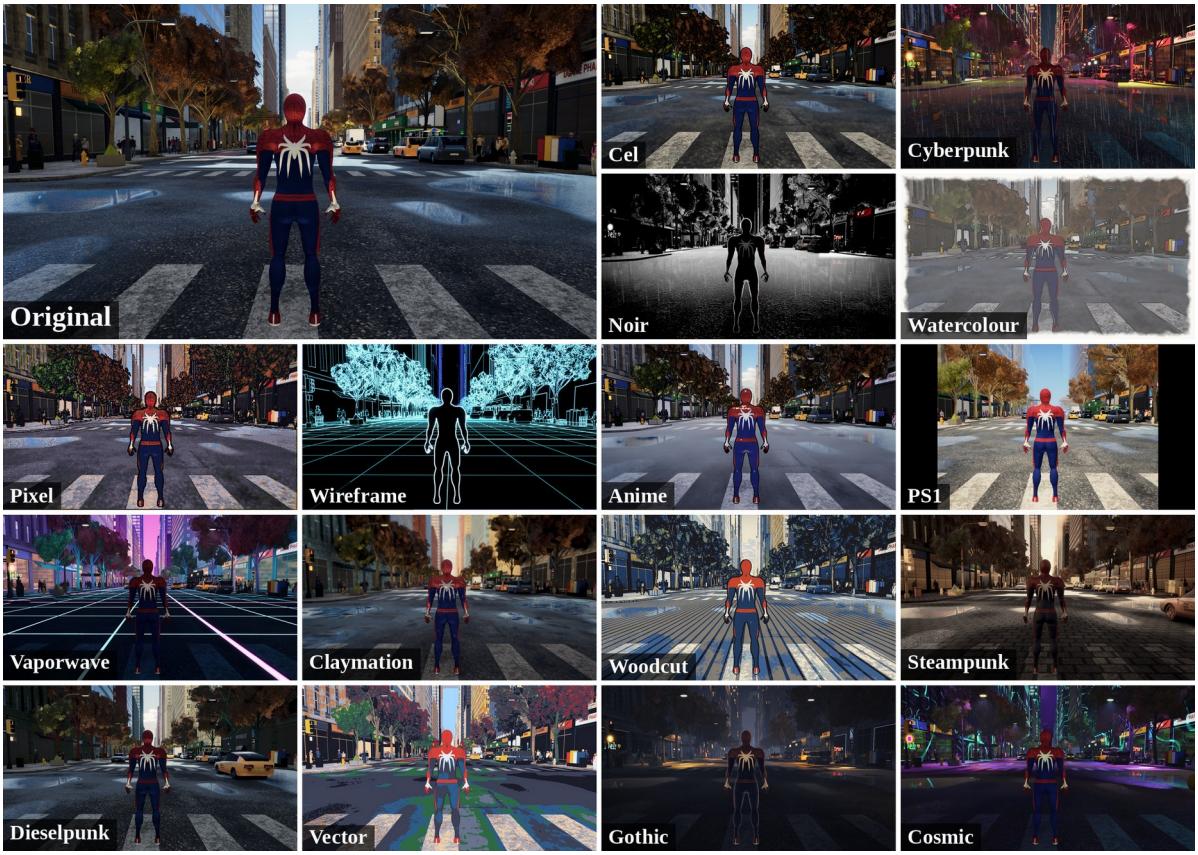


Figure C.1: Street scene: the original frame (top left) and 16 visual themes

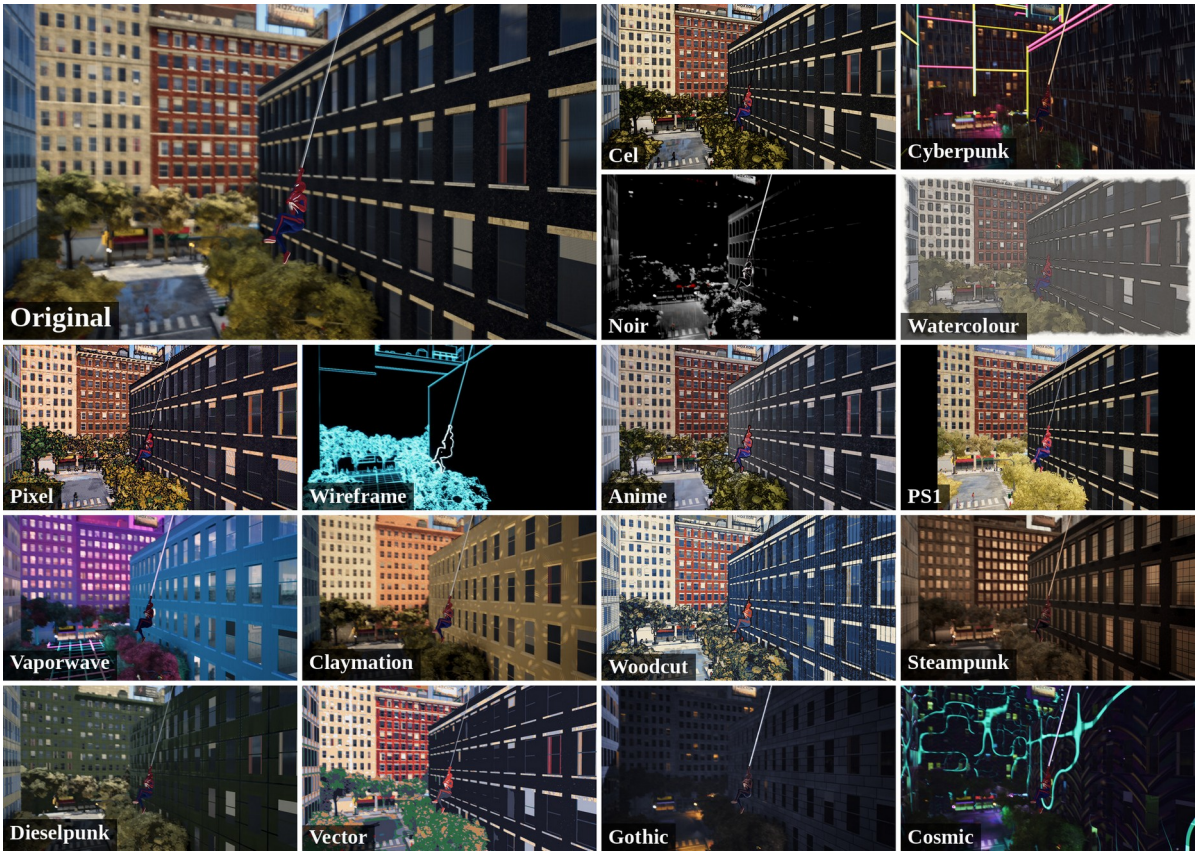


Figure C.2: Swinging scene: the original frame (top left) and 16 visual themes